BattleChain VulnerableVault Audit Report

Version 1.0

alva-p.xyz

March 27, 2026

- Prepared by: alva-p
- Lead Auditors: Alvaro Pineda

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Informational

Protocol Summary

BattleChain is a Cyfrin educational testnet that allows legal exploitation of intentionally vulnerable smart contracts under a Safe Harbor agreement. The VulnerableVault protocol is a simple ERC-20 token vault that allows users to deposit and withdraw BattleChain Demo Tokens (BCDT). The protocol's stated functions are:

1. Deposit tokens into the vault via `deposit(uint256 amount)`.
2. Withdraw all deposited tokens via `withdrawAll()`.
3. Query deposited balance via `getBalance(address user)`.

The BCDT token (`MockToken`) supports an optional per-address transfer hook: any contract can register itself via `setTransferHook(address)` to receive an `onTokenTransfer(address from, uint256 amount)` callback whenever it receives tokens.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibility for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts. This audit was performed in the context of the BattleChain Safe Harbor program — all exploits were executed on a public testnet designed for this purpose.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity.

Audit Details

- Commit Hash: d532f01 (battlechain-lib: 40cf778)
- Chain: BattleChain Testnet (chainId: 627)
- Block of exploit TX: 2163
- TX Hash: `0x93c77ae19fca33d0c3a7afb54e94b1131fcffbd8a7706a367593432292f257a8`

Scope

```
./src/
  VulnerableVault.sol
  MockToken.sol
```

Roles

- **Depositor** — Any address may deposit BCDT into the vault and later withdraw their balance.
- **Hook Registrant** — Any address may register a transfer hook contract on MockToken.
- **Minter** — Anyone can call `MockToken.mint()` to create arbitrary BCDT (intentionally unconstrained for this tutorial).

Contracts

Contract	Address
MockToken (BCDT)	`0xA73C122D7D905de671217b7a1a357a4Cb3441478`
VulnerableVault	`0x775aD4B8f11Fc3F7B83B99331e770A3131C65ceF`
Attacker	`0xDe8e1e159ac6D16BDfc4f1A8DcF36a18dc61f9e1`
Auditor address	`0xe482E762e6430Ce2E43258A4681040786675d167`

Executive Summary

A complete audit of `VulnerableVault.sol` and `MockToken.sol` was performed on March 27, 2026. One critical reentrancy vulnerability was identified and successfully demonstrated on-chain via a white-hat exploit under BattleChain Safe Harbor. The vault was fully drained (1100 BCDT), and 90% of recovered funds (990 BCDT) were returned to the designated recovery address, with a 10% bounty (110 BCDT) retained per Safe Harbor terms.

Additionally, two secondary issues were identified in `MockToken.sol`: an unrestricted mint function and a transfer hook mechanism that, while intentional for this tutorial, constitute meaningful attack surface in any production context.

Issues found

Severity	Number of issues found
High	1
Medium	1

Severity	Number of issues found
Informational	1
Total	3

Findings

High

[H-1] Reentrancy in `VulnerableVault::withdrawAll` via token transfer hook enables complete vault drain

Description: `VulnerableVault::withdrawAll` violates the Checks-Effects-Interactions (CEI) pattern. The function transfers tokens to the caller (`Interaction`) before zeroing the caller's balance (`Effect`). If the token invokes a callback on the recipient during `transfer()`, an attacker can re-enter `withdrawAll()` before the balance is cleared — each recursive call sees the original non-zero balance and succeeds.

```
function withdrawAll() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

    // INTERACTION before EFFECT
@>  TOKEN.transfer(msg.sender, amount); // triggers onTokenTransfer hook

    // Effect is too late     re-entrant calls already passed the require above
@>  balances[msg.sender] = 0;

    emit Withdrawn(msg.sender, amount);
}
```

`MockToken.transfer()` calls `onTokenTransfer(from, amount)` on any hook contract registered for the recipient. By registering the attacker contract as its own hook via `setTransferHook`, the attacker controls a callback that fires on every incoming transfer — before the vault updates its state.

Impact: The entire vault balance can be drained in a single transaction. Any depositor's funds are at risk.

Proof of Concept:

On-chain execution — block 2163, BattleChain Testnet:

```
TX: 0x93c77ae19fca33d0c3a7afb54e94b1131fcffbd8a7706a367593432292f257a8
Gas used: 253,664 / 500,000
Status: Success
```

Event log sequence (28 logs total):

Log index	Event	Details
0x00	`TransferHookSet`	Attacker registers itself as hook
0x01	`Transfer(0x0 -> Attacker)`	Mint 100 BCDT seed
0x02	`Approval(Attacker -> Vault)`	Approve vault for deposit
0x03	`Transfer(Attacker -> Vault)`	Deposit 100 BCDT seed
0x04	`Deposited(Attacker, 100e18)`	Vault confirms deposit
0x05..0x0f	`Transfer(Vault -> Attacker)` x11	11 reentrancy iterations, 100 BCDT each
0x10..0x1a	`Withdrawn(Attacker, 100e18)` x11	Vault emits 11 withdrawal events
0x1b	`Transfer(Attacker -> 0xe482...)`	990 BCDT returned to recovery (90%)
0x1c	`Transfer(Attacker -> 0xe482...)`	110 BCDT bounty to auditor (10%)

Attack flow:

1. Attacker calls `token.setTransferHook(address(this))` — registers reentrancy callback.

2. Attacker calls `token.mint(address(this), 100e18)` — obtains seed tokens at zero cost.
3. Attacker calls `vault.deposit(100e18)` — establishes a legitimate vault balance.
4. Attacker calls `vault.withdrawAll()`:
   - Vault reads `balances[attacker] = 100e18`, passes `require`.
   - Vault calls `token.transfer(attacker, 100e18)`.
   - MockToken calls `attacker.onTokenTransfer()` before returning.
   - Attacker re-enters `vault.withdrawAll()`.
   - `balances[attacker]` still equals `100e18` (not yet zeroed).
   - Loop repeats until vault balance is 0.
5. 11 recursive withdrawals drain all 1100 BCDT (1000 pre-existing + 100 seed).

```
// Attacker.sol     re-entry point
function onTokenTransfer(address, uint256) external {
    if (TOKEN.balanceOf(address(VAULT)) > 0) {
        VAULT.withdrawAll(); // re-enters before balances[msg.sender] =
            0
    }
}
```

Recommended Mitigation: Apply CEI strictly — zero the balance before transferring:

```
function withdrawAll() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

+   balances[msg.sender] = 0;
+   emit Withdrawn(msg.sender, amount);

    TOKEN.transfer(msg.sender, amount);

-   balances[msg.sender] = 0;
-   emit Withdrawn(msg.sender, amount);
}
```

Alternatively, add a `nonReentrant` modifier (e.g., OpenZeppelin `ReentrancyGuard`). Both fixes are recommended together.

Medium

[M-1] `MockToken::mint` is unrestricted — anyone can mint arbitrary tokens

Description: The `mint(address to, uint256 amount)` function in `MockToken` has no access control. Any externally owned account or contract can mint unlimited tokens to any address.

```
function mint(address to, uint256 amount) external {
    _mint(to, amount); // no access check
}
```

Impact: In any production deployment, this allows infinite token creation, making the token valueless and enabling trivially funded reentrancy attacks (as demonstrated — the attacker minted its own 100 BCDT seed with zero cost).

Note: This is intentional in the BattleChain tutorial context. In a production ERC-20, mint access must be restricted to authorized roles only.

Recommended Mitigation:

```
+ address public immutable MINTER;
+ constructor() ERC20("BattleChain Demo Token", "BCDT") {
+     MINTER = msg.sender;
+ }

  function mint(address to, uint256 amount) external {
+     require(msg.sender == MINTER, "not authorized");
      _mint(to, amount);
  }
```

Informational

[I-1] `MockToken::transfer` hook mechanism introduces uncontrolled re-entry vectors

Description: `MockToken` overrides `transfer()` to call an arbitrary external contract registered by the recipient via `setTransferHook()`. This means any ERC-20 `transfer` to a hooked address becomes a potential re-entry point for every protocol built on top of this token.

```
function transfer(address to, uint256 amount) public override returns (
    bool) {
     bool success = super.transfer(to, amount);
     address hook = transferHooks[to];
     if (hook != address(0)) {
```

```
        ITransferHook(hook).onTokenTransfer(msg.sender, amount); // arbitrary external call
    }
    return success;
}
```

Impact: Any vault, AMM, or protocol that holds this token and calls `transfer()` on it is potentially vulnerable to reentrancy if the recipient has a malicious hook registered. This is an emergent property that developers integrating the token may not anticipate.

Recommended Mitigation: If the hook pattern is intentional, document it prominently and require all protocols integrating the token to audit their own CEI compliance. Consider emitting an event rather than calling an external contract, or restricting hook registration to a trusted registry.

Safe Harbor Summary

This audit was performed under the BattleChain Safe Harbor program. Upon successful exploitation:

- **Total drained:** 1100 BCDT (1000 BCDT pre-existing vault balance + 100 BCDT attacker seed)
- **Reentrancy depth:** 11 iterations
- **Returned to recovery:** 990 BCDT (90%) → `0xe482E762e6430Ce2E43258A4681040786675d167`
- **Auditor bounty:** 110 BCDT (10%) → `0xe482E762e6430Ce2E43258A4681040786675d167`
- **Exploit TX:** `0x93c77ae...257a8`
- **Block:** 2163 | **Chain:** BattleChain Testnet (chainId 627)